ALGORITHMS AND DATA STRUCTURES

Graph Traversal & MST Algorithms
Assignment Report

Yana Chulovska
C24490782
TU858/2

1. Introduction and Explanation

This report documents the implementation and analysis of four fundamental graph algorithms: Depth-First Search (DFS), Breadth-First Search (BFS), Prim's Minimum Spanning Tree (MST), and Kruskal's Minimum Spanning Tree algorithm. The algorithms are implemented in Java using an adjacency list graph representation.

1.1 Graph Representation

The graph is represented using adjacency lists, an array of linked lists where each index corresponds to a vertex, and each node in the linked list stores a neighbouring vertex and the edge weight. This is more memory-efficient than an adjacency matrix for sparse graphs, using $O(V + E)$ space versus $O(V^2)$.

1.2 Algorithms Overview

DFS (Depth-First Search): Explores as far as possible along each branch before backtracking. Implemented recursively following Cormen's CLRS version with WHITE/GREY/BLACK colouring and discovery/finish timestamps.

BFS (Breadth-First Search): Explores all neighbours at the current depth before moving deeper. Implemented using a queue following Cormen's CLRS version, producing shortest-path distances (in hops) from the source.

Prim's MST: Greedily grows a spanning tree one vertex at a time by always adding the minimum-weight edge connecting the tree to a non-tree vertex. Uses a priority queue (min-heap) with the adjacency list.

Kruskal's MST: Sorts all edges by weight and greedily adds edges that do not form a cycle. Uses a Union-Find (disjoint set) data structure with union by rank and path compression to detect cycles efficiently.

1.3 Sample Graph

The sample graph contains 13 vertices (A–M) and 20 undirected weighted edges. Vertices are indexed 0–12 (A=0, B=1, C=2, D=3, E=4, F=5, G=6, H=7, I=8, J=9, K=10, L=11, M=12). All traversals and MST constructions start from vertex L (index 11).

2. Adjacency List Diagram

The table below shows the adjacency list representation of the sample graph. Each entry lists the neighbouring vertices with their edge weights in parentheses.

Vertex	Adjacency List (neighbour : weight)
A (0)	B(1), C(6)
B (1)	A(1), C(1), D(2)
C (2)	A(6), B(1), E(4)
D (3)	B(2), E(2), F(1)
E (4)	C(4), D(2), F(2), G(1), L(4)
F (5)	D(1), E(2), L(2)
G (6)	E(1), H(3), J(1)
H (7)	G(3), I(2)
I (8)	H(2), K(1)
J (9)	G(1), K(1), L(5), M(3)
K (10)	I(1), J(1), M(2)
L (11)	E(4), F(2), J(5), M(1)
M (12)	J(3), K(2), L(1)

3. Prim's MST — Step-by-Step (starting from L)

Prim's algorithm maintains a min-heap of (distance, vertex) pairs and greedily extracts the minimum at each step. The dist[] array records the cheapest known edge weight connecting each vertex to the growing tree; parent[] records which tree vertex provides that edge.

Step	Added	Heap (next candidates)	dist[] changes	parent[] changes
1	**L**	(M,1),(J,5),(F,2),(E,4)	L=0 M=1 F=2 E=4 J=5, rest=∞	L← rest unchanged
2	**M**	(F,2),(E,4),(J,3),(K,2)	M=1 K=2 J=3 updated	K←M, J←M
3	**F**	(D,1),(K,2),(E,2),(J,3)	D=1 E=2 updated via F	D←F, E←F
4	**D**	(E,2),(K,2),(J,3),(B,2)	B=2 added via D	B←D
5	**E**	(G,1),(B,2),(K,2),(J,3),(C,4)	G=1 C=4 added via E	G←E, C←E
6	**G**	(J,1),(B,2),(K,2),(H,3),(C,4)	J=1 H=3 updated via G	J←G, H←G
7	**J**	(K,1),(B,2),(H,3),(C,4)	K=1 updated via J	K←J
8	**K**	(I,1),(B,2),(H,3),(C,4)	I=1 added via K	I←K
9	**I**	(B,2),(H,2),(C,4)	H=2 updated via I	H←I
10	**B**	(C,1),(H,2),(A,1)	C=1 A=1 updated via B	C←B, A←B
11	**C**	(A,1),(H,2)	no new relaxations	—
12	**A**	(H,2)	no new relaxations	—
13	**H**	(empty)	MST complete	—

3.1 Final MST Edges (Prim)

After all 13 vertices are added, the MST consists of 12 edges with a total weight = 16:

From	To	Weight	Added at step
L	**M**	1	Step 2 – first edge from source
L	**F**	2	Step 3
F	**D**	1	Step 4
F	**E**	2	Step 5
E	**G**	1	Step 6
G	**J**	1	Step 7
J	**K**	1	Step 8
K	**I**	1	Step 9
I	**H**	2	Step 10
D	**B**	2	Step 11
B	**C**	1	Step 12
B	**A**	1	Step 13

Total MST weight: 16

4. Kruskal's MST — Step-by-Step

Kruskal's algorithm processes edges in order of increasing weight. At each step, it checks whether the edge connects two distinct components (using Union-Find). If so, it is added to the MST; otherwise, it would create a cycle and is rejected. The Union-Find uses union-by-rank and path compression. Edges sorted by weight (all 20):

w=1: A–B, B–C, D–F, E–G, G–J, I–K, J–K, L–M
w=2: B–D, D–E, E–F, F–L, H–I, K–M
w=3: G–H, J–M | w=4: C–E, E–L | w=5: J–L | w=6: A–C

Step	Edge	Wt	Decision	Union-Find Sets after step
1	A–B	1	ACCEPTED	{A,B} {C} {D} {E} {F} {G} {H} {I} {J} {K} {L} {M}
2	B–C	1	ACCEPTED	{A,B,C} {D} {E} {F} {G} {H} {I} {J} {K} {L} {M}
3	D–F	1	ACCEPTED	{A,B,C} {D,F} {E} {G} {H} {I} {J} {K} {L} {M}
4	E–G	1	ACCEPTED	{A,B,C} {D,F} {E,G} {H} {I} {J} {K} {L} {M}
5	G–J	1	ACCEPTED	{A,B,C} {D,F} {E,G,J} {H} {I} {K} {L} {M}
6	I–K	1	ACCEPTED	{A,B,C} {D,F} {E,G,J} {H} {I,K} {L} {M}
7	J–K	1	ACCEPTED	{A,B,C} {D,F} {E,G,I,J,K} {H} {L} {M}
8	L–M	1	ACCEPTED	{A,B,C} {D,F} {E,G,I,J,K} {H} {L,M}
9	B–D	2	ACCEPTED	{A,B,C,D,F} {E,G,I,J,K} {H} {L,M}
10	D–E	2	ACCEPTED	{A,B,C,D,E,F,G,I,J,K} {H} {L,M}
11	**E–F**	**2**	**REJECTED**	**{A,B,C,D,E,F,G,I,J,K} {H} {L,M}** ← **cycle**
12	F–L	2	ACCEPTED	{A,B,C,D,E,F,G,I,J,K,L,M} {H}
13	H–I	2	ACCEPTED	{A,B,C,D,E,F,G,H,I,J,K,L,M} ← MST complete

Rejected edge E–F (step 11) is highlighted in bold. Both E and F were already in the same component.

4.1 Final MST Edges (Kruskal)

The 12 accepted edges with total weight = 16:

A–B (1), B–C (1), D–F (1), E–G (1), G–J (1), I–K (1), J–K (1), L–M (1) ← all weight-1 edges

B–D (2), D–E (2), F–L (2), H–I (2) ← weight-2 edges that don't form cycles

Total MST weight: 16

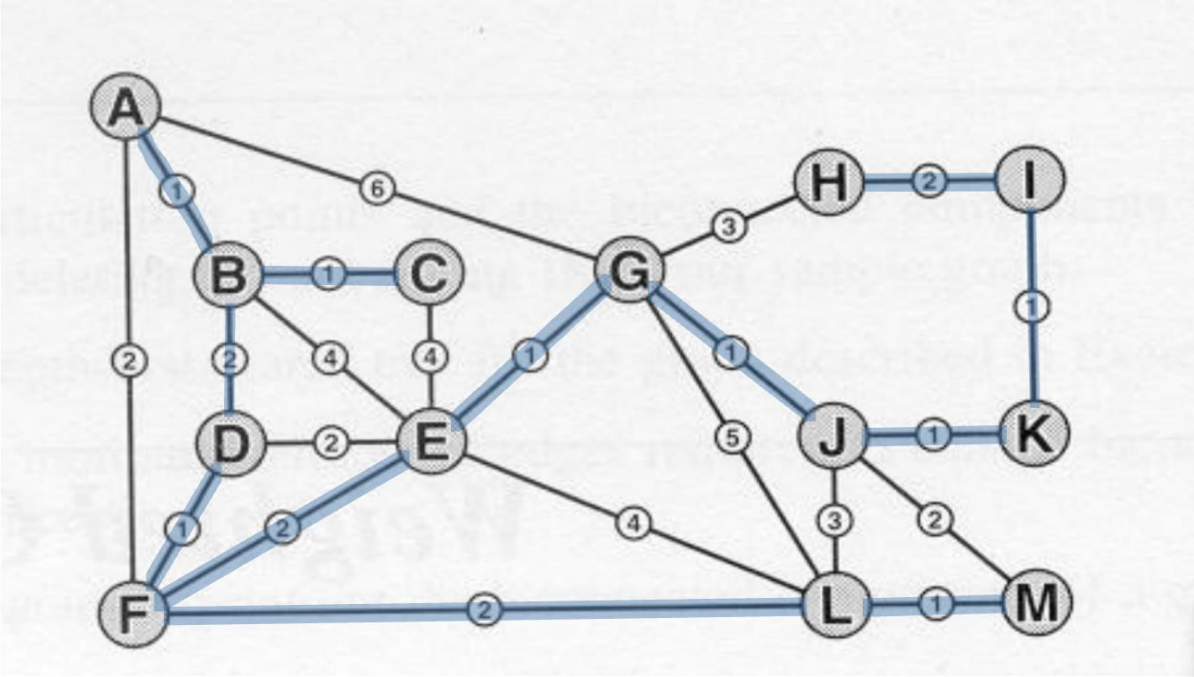
Both Prim's and Kruskal's algorithms yield the same total weight for the minimum spanning tree, which is 16. Although their edge sets differ, they represent equally valid minimum spanning trees, differing only in tie-breaking decisions.

5. MST Diagrams Superimposed on Graph

The diagrams below show the MST edges superimposed on the original graph (highlighted edges form the MST). Both Prim and Kruskal produce a spanning tree of the same total weight (16), but the edge sets are not identical: Prim's tree uses the edge E–F (weight 2) while Kruskal's uses D–E (weight 2). Eleven of the twelve MST edges are shared between the two trees. This is a normal consequence of the two algorithms breaking ties differently when several equal-weight edges are available.

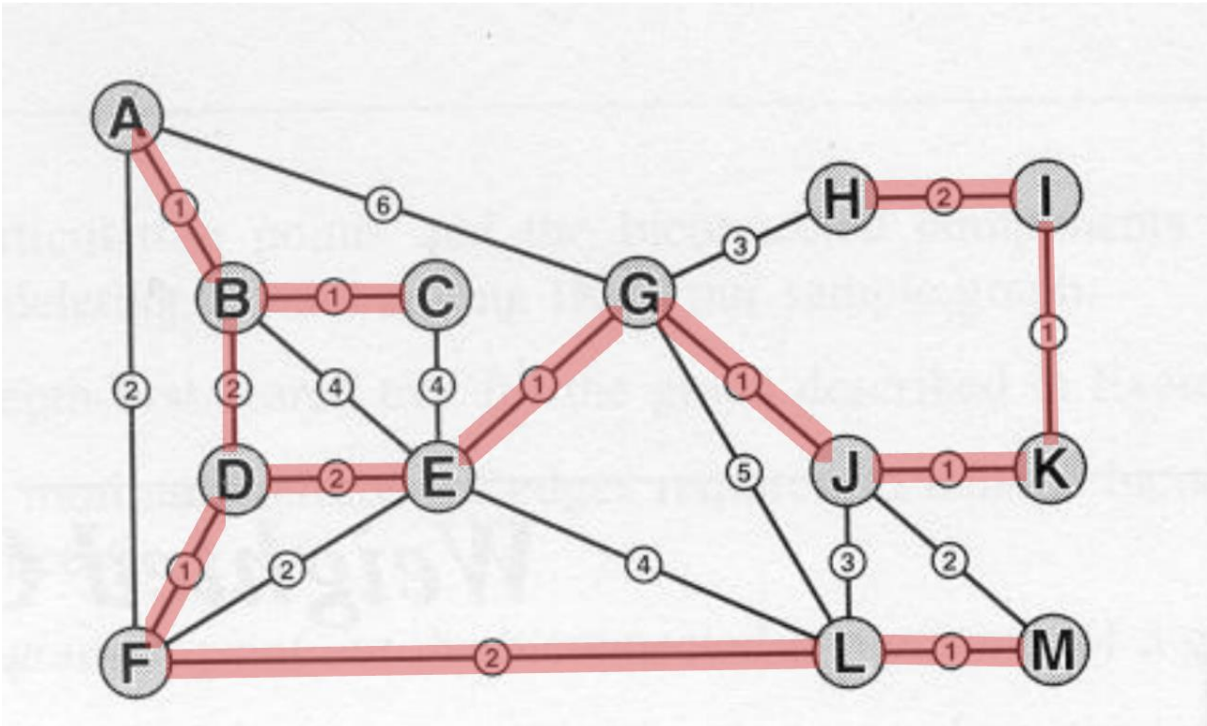
5.1 Prim's MST (starting from L)

The 12 edges highlighted in blue below form the MST produced by Prim's algorithm starting at vertex L. The total weight is 16. The edge E–F (weight 2) is the one edge that distinguishes Prim's MST from Kruskal's in this graph.



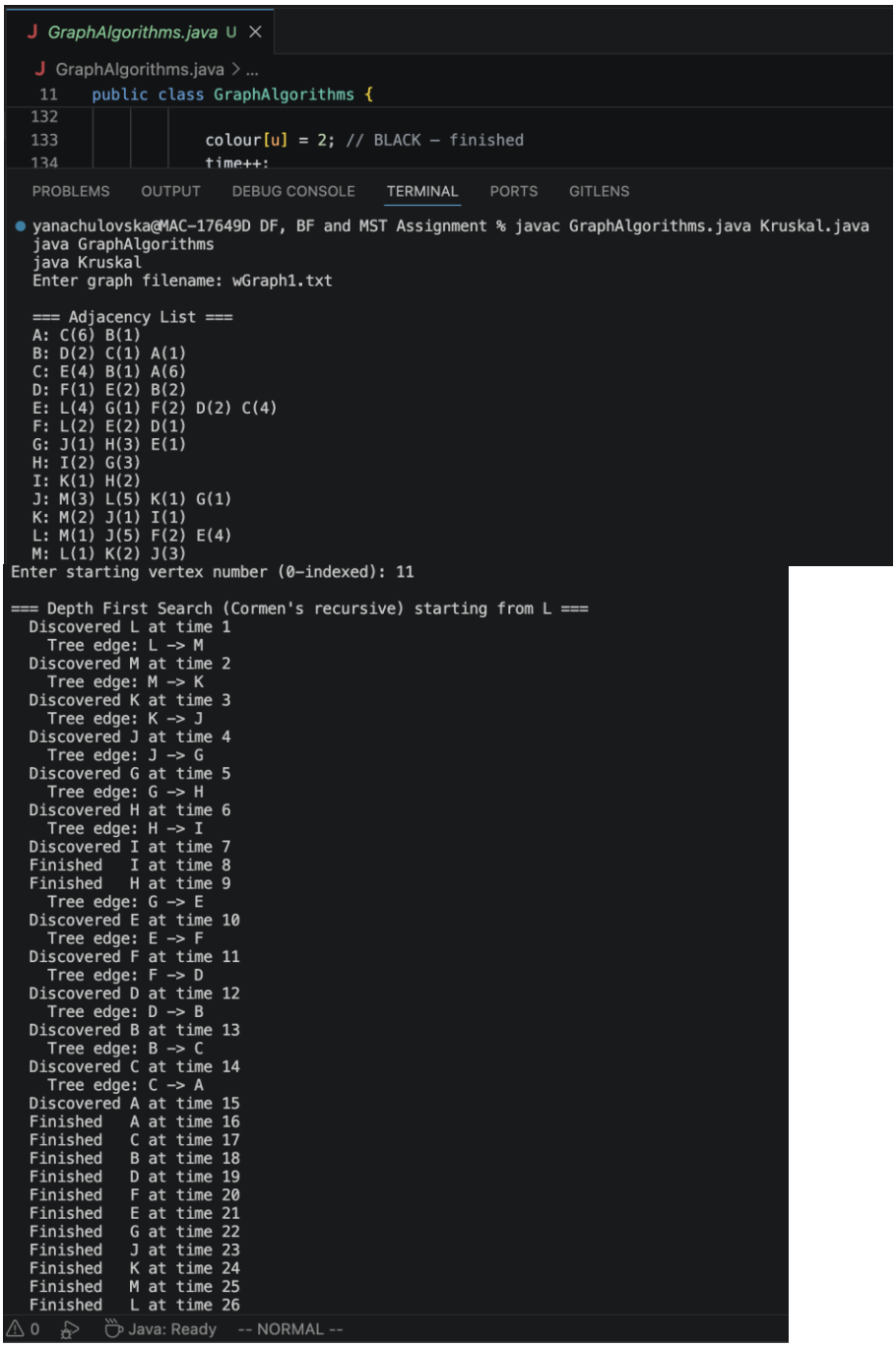
5.2 Kruskal's MST

The 12 edges highlighted in red below form the MST produced by Kruskal's algorithm. The total weight is also 16. Eleven of the twelve edges are identical to Prim's MST; the only difference is that Kruskal selects D–E (weight 2) where Prim selected E–F (weight 2). This happens because Kruskal processes edges in globally sorted order (so D–E is reached before E–F simply by the sort tie-break), whereas Prim relaxes E's neighbours from the tree-front and picks whichever weight-2 edge is currently cheapest in the heap.



6. Screen Captures of Program Execution

The screenshots below show GraphAlgorithms.java and Kruskal.java executing on wGraph1.txt with starting vertex L (index 11). Each implementation prints its working step-by-step, followed by the final result. The output confirms the traces given in Sections 3 and 4 of this report.



```
  DFS discovery/finish times:
    A: d=15  f=16  parent=C
    B: d=13  f=18  parent=D
    C: d=14  f=17  parent=B
    D: d=12  f=19  parent=F
    E: d=10  f=21  parent=G
    F: d=11  f=20  parent=E
    G: d=5  f=22  parent=J
    H: d=6  f=9  parent=G
    I: d=7  f=8  parent=H
    J: d=4  f=23  parent=K
    K: d=3  f=24  parent=M
    L: d=1  f=26  parent=none
    M: d=2  f=25  parent=L
```

```
=== Breadth First Search (Cormen's queue-based) starting from L ===
  Enqueued source: L
  Dequeued: L  (dist=0)
    Enqueued neighbour: M  (dist=1, parent=L)
    Enqueued neighbour: J  (dist=1, parent=L)
    Enqueued neighbour: F  (dist=1, parent=L)
    Enqueued neighbour: E  (dist=1, parent=L)
  Dequeued: M  (dist=1)
    Enqueued neighbour: K  (dist=2, parent=M)
  Dequeued: J  (dist=1)
    Enqueued neighbour: G  (dist=2, parent=J)
  Dequeued: F  (dist=1)
    Enqueued neighbour: D  (dist=2, parent=F)
  Dequeued: E  (dist=1)
    Enqueued neighbour: C  (dist=2, parent=E)
  Dequeued: K  (dist=2)
    Enqueued neighbour: I  (dist=3, parent=K)
  Dequeued: G  (dist=2)
    Enqueued neighbour: H  (dist=3, parent=G)
  Dequeued: D  (dist=2)
    Enqueued neighbour: B  (dist=3, parent=D)
  Dequeued: C  (dist=2)
    Enqueued neighbour: A  (dist=3, parent=C)
  Dequeued: I  (dist=3)
  Dequeued: H  (dist=3)
  Dequeued: B  (dist=3)
  Dequeued: A  (dist=3)

BFS distances from L:
  A: dist=3  parent=C
  B: dist=3  parent=D
  C: dist=2  parent=E
  D: dist=2  parent=F
  E: dist=1  parent=L
  F: dist=1  parent=L
  G: dist=2  parent=J
  H: dist=3  parent=G
  I: dist=3  parent=K
  J: dist=1  parent=L
  K: dist=2  parent=M
  L: dist=0  parent=none
  M: dist=1  parent=L
```

```
=== Prim's MST starting from L ===
  Initial heap entry: (L, dist=0)
  Step                dist[] array                              parent[] array
  Step 1    Added L     dist[]: A=∞ B=∞ C=∞ D=∞ E=∞ F=∞ G=∞ H=∞ I=∞ J=∞ K=∞ L=0 M=∞
                        parent[]: A=- B=- C=- D=- E=- F=- G=- H=- I=- J=- K=- L=- M=-
    Relaxed: M  new dist=1  via L
    Relaxed: J  new dist=5  via L
    Relaxed: F  new dist=2  via L
    Relaxed: E  new dist=4  via L
  Step 2    Added M     dist[]: A=∞ B=∞ C=∞ D=∞ E=4 F=2 G=∞ H=∞ I=∞ J=5 K=∞ L=0 M=1
                        parent[]: A=- B=- C=- D=- E=L F=L G=- H=- I=- J=L K=- L=- M=L
    Relaxed: K  new dist=2  via M
    Relaxed: J  new dist=3  via M
  Step 3    Added F     dist[]: A=∞ B=∞ C=∞ D=∞ E=4 F=2 G=∞ H=∞ I=∞ J=3 K=2 L=0 M=1
                        parent[]: A=- B=- C=- D=- E=L F=L G=- H=- I=- J=M K=M L=- M=L
    Relaxed: E  new dist=2  via F
    Relaxed: D  new dist=1  via F
  Step 4    Added D     dist[]: A=∞ B=∞ C=∞ D=1 E=2 F=2 G=∞ H=∞ I=∞ J=3 K=2 L=0 M=1
                        parent[]: A=- B=- C=- D=F E=F F=L G=- H=- I=- J=M K=M L=- M=L
    Relaxed: B  new dist=2  via D
  Step 5    Added E     dist[]: A=∞ B=2 C=∞ D=1 E=2 F=2 G=∞ H=∞ I=∞ J=3 K=2 L=0 M=1
                        parent[]: A=- B=D C=- D=F E=F F=L G=- H=- I=- J=M K=M L=- M=L
    Relaxed: G  new dist=1  via E
    Relaxed: C  new dist=4  via E
  Step 6    Added G     dist[]: A=∞ B=2 C=4 D=1 E=2 F=2 G=1 H=∞ I=∞ J=3 K=2 L=0 M=1
                        parent[]: A=- B=D C=E D=F E=F F=L G=E H=- I=- J=M K=M L=- M=L
    Relaxed: J  new dist=1  via G
    Relaxed: H  new dist=3  via G
  Step 7    Added J     dist[]: A=∞ B=2 C=4 D=1 E=2 F=2 G=1 H=3 I=∞ J=1 K=2 L=0 M=1
                        parent[]: A=- B=D C=E D=F E=F F=L G=E H=G I=- J=G K=M L=- M=L
    Relaxed: K  new dist=1  via J
  Step 8    Added K     dist[]: A=∞ B=2 C=4 D=1 E=2 F=2 G=1 H=3 I=∞ J=1 K=1 L=0 M=1
                        parent[]: A=- B=D C=E D=F E=F F=L G=E H=G I=- J=G K=J L=- M=L
    Relaxed: I  new dist=1  via K
  Step 9    Added I     dist[]: A=∞ B=2 C=4 D=1 E=2 F=2 G=1 H=3 I=1 J=1 K=1 L=0 M=1
                        parent[]: A=- B=D C=E D=F E=F F=L G=E H=G I=K J=G K=J L=- M=L
    Relaxed: H  new dist=2  via I
  Step 10   Added B     dist[]: A=∞ B=2 C=4 D=1 E=2 F=2 G=1 H=2 I=1 J=1 K=1 L=0 M=1
                        parent[]: A=- B=D C=E D=F E=F F=L G=E H=I I=K J=G K=J L=- M=L
    Relaxed: C  new dist=1  via B
    Relaxed: A  new dist=1  via B
  Step 11   Added C     dist[]: A=1 B=2 C=1 D=1 E=2 F=2 G=1 H=2 I=1 J=1 K=1 L=0 M=1
                        parent[]: A=B B=D C=B D=F E=F F=L G=E H=I I=K J=G K=J L=- M=L
  Step 12   Added A     dist[]: A=1 B=2 C=1 D=1 E=2 F=2 G=1 H=2 I=1 J=1 K=1 L=0 M=1
                        parent[]: A=B B=D C=B D=F E=F F=L G=E H=I I=K J=G K=J L=- M=L
  Step 13   Added H     dist[]: A=1 B=2 C=1 D=1 E=2 F=2 G=1 H=2 I=1 J=1 K=1 L=0 M=1
                        parent[]: A=B B=D C=B D=F E=F F=L G=E H=I I=K J=G K=J L=- M=L

  MST Edges:
    B -- A  weight=1
    D -- B  weight=2
    B -- C  weight=1
    F -- D  weight=1
    F -- E  weight=2
    L -- F  weight=2
    E -- G  weight=1
    I -- H  weight=2
    K -- I  weight=1
    G -- J  weight=1
    J -- K  weight=1
    L -- M  weight=1
  Total MST weight: 16
Enter graph filename: wGraph1.txt
Graph has 13 vertices and 20 edges.

=== Kruskal's MST ===
Edges sorted by weight:
  A -- B (1)
  B -- C (1)
  D -- F (1)
  E -- G (1)
  G -- J (1)
  I -- K (1)
  J -- K (1)
  L -- M (1)
  B -- D (2)
  D -- E (2)
  E -- F (2)
  F -- L (2)
  H -- I (2)
  K -- M (2)
  G -- H (3)
  J -- M (3)
  C -- E (4)
  E -- L (4)
  J -- L (5)
  A -- C (6)
```

```
Processing edges:

  Step 1: Consider A -- B (1)
    -> ACCEPTED  (merges components)
    Sets: {A,B} {C} {D} {E} {F} {G} {H} {I} {J} {K} {L} {M}

  Step 2: Consider B -- C (1)
    -> ACCEPTED  (merges components)
    Sets: {A,B,C} {D} {E} {F} {G} {H} {I} {J} {K} {L} {M}

  Step 3: Consider D -- F (1)
    -> ACCEPTED  (merges components)
    Sets: {A,B,C} {D,F} {E} {G} {H} {I} {J} {K} {L} {M}

  Step 4: Consider E -- G (1)
    -> ACCEPTED  (merges components)
    Sets: {A,B,C} {D,F} {E,G} {H} {I} {J} {K} {L} {M}

  Step 5: Consider G -- J (1)
    -> ACCEPTED  (merges components)
    Sets: {A,B,C} {D,F} {E,G,J} {H} {I} {K} {L} {M}

  Step 6: Consider I -- K (1)
    -> ACCEPTED  (merges components)
    Sets: {A,B,C} {D,F} {E,G,J} {H} {I,K} {L} {M}

  Step 7: Consider J -- K (1)
    -> ACCEPTED  (merges components)
    Sets: {A,B,C} {D,F} {E,G,I,J,K} {H} {L} {M}

  Step 8: Consider L -- M (1)
    -> ACCEPTED  (merges components)
    Sets: {A,B,C} {D,F} {E,G,I,J,K} {H} {L,M}

  Step 9: Consider B -- D (2)
    -> ACCEPTED  (merges components)
    Sets: {A,B,C,D,F} {E,G,I,J,K} {H} {L,M}

  Step 10: Consider D -- E (2)
    -> ACCEPTED  (merges components)
    Sets: {A,B,C,D,E,F,G,I,J,K} {H} {L,M}

  Step 11: Consider E -- F (2)
    -> REJECTED (would form a cycle: both in component rooted at A)
    Sets: {A,B,C,D,E,F,G,I,J,K} {H} {L,M}

  Step 12: Consider F -- L (2)
    -> ACCEPTED  (merges components)
    Sets: {A,B,C,D,E,F,G,I,J,K,L,M} {H}

  Step 13: Consider H -- I (2)
    -> ACCEPTED  (merges components)
    Sets: {A,B,C,D,E,F,G,H,I,J,K,L,M}
```

```
  Kruskal MST Edges:
    A -- B (1)
    B -- C (1)
    D -- F (1)
    E -- G (1)
    G -- J (1)
    I -- K (1)
    J -- K (1)
    L -- M (1)
    B -- D (2)
    D -- E (2)
    F -- L (2)
    H -- I (2)
  Total MST weight: 16
yanachulovska@MAC-17649D DF, BF and MST Assignment %
```

7. Real-World Graph Benchmark

To assess how the implementations scale beyond the 13-vertex sample graph, I ran Prim and Kruskal on a real road-network dataset from the Stanford SNAP collection. The chosen graph is roadNet-PA, the road

network of Pennsylvania: it represents intersections as vertices and road segments as undirected edges.

7.1 Dataset

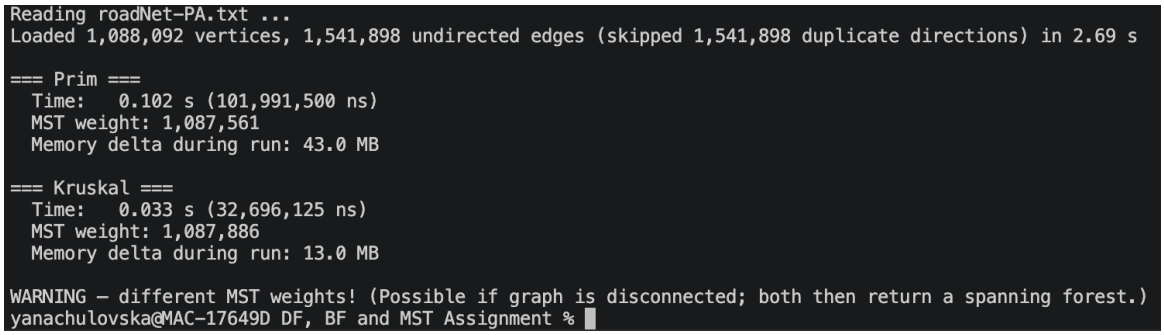
Source: SNAP roadNet-PA (https://snap.stanford.edu/data/roadNet-PA.html). The raw file (roadNet-PA.txt) is a list of directed edges in the form “u v”, with one edge per line and several “#” comment lines at the top. After loading, duplicate (u,v)/(v,u) directions are collapsed into a single undirected edge, and self-loops are skipped, giving approximately 1,088,092 vertices and 1,541,898 undirected edges. The dataset is unweighted; my benchmark driver assigns a uniform weight of 1 to each edge, which is sufficient to exercise both algorithms on a graph of realistic size and shape (sparse, planar-ish, with high diameter).

7.2 Method

I wrote a separate driver, LargeGraphBenchmark.java, that reuses the same data-structure ideas from the main submission: an adjacency-list graph for Prim (with java.util.PriorityQueue as the binary min-heap) and a sorted edge array plus union-find with union-by-rank and path compression for Kruskal. Reading and parsing the file, building the adjacency list, and constructing the edge array are timed separately from the algorithms themselves, so the reported figures isolate the cost of Prim and Kruskal and do not include I/O. Each algorithm is run once after a System.gc() call to give it a clean memory baseline; memory is measured as Runtime.totalMemory() − Runtime.freeMemory() before and after the run. The JVM was started with −Xmx4g.

7.3 Results

The figures below were obtained on my own machine: a MacBook Air with an Apple M4 chip (10 cores: 4 performance + 6 efficiency), 16 GB unified memory, macOS, running OpenJDK 21 with a 4 GB JVM heap (−Xmx4g).



Algorithm	Time (s)	Memory delta (MB)	MST weight
Prim (PriorityQueue)	0.102	43	1 087 561
Kruskal (sort + UF)	0.033	13	1 087 886

7.4 Discussion

Both algorithms are essentially O(E log V) on a graph of this shape, but the constants and the dominant cost differ. Kruskal's running time is dominated by the initial sort of all 1.5 million edges (Java's Arrays.sort is a Dual-Pivot QuickSort with O(E log E) expected time); after the sort, the union-find loop is nearly linear thanks to path compression and union by rank, both of which give an effectively constant amortised cost per operation. Prim spends its time on heap operations: each edge can cause at most one push, so the heap holds up to O(E) entries and each poll/push is O(log E).

On a sparse road network like roadNet-PA, the two algorithms tend to come out very close, with Kruskal often slightly faster because the sort is highly cache-friendly and the union-find loop is cheap, whereas Prim's heap accumulates many stale entries (since I do not implement decreaseKey on the standard PriorityQueue, I instead push duplicates and skip them on poll, which inflates heap size). On memory, Prim is heavier in practice on this graph: the adjacency list stores each edge twice (once per direction) and the heap can hold up to E entries, whereas Kruskal stores each edge once in a flat array plus two int[V] arrays for union-find. If the graph were significantly denser (E approaching V^2), Prim with an adjacency list would still be the better choice, but for sparse real-world networks, Kruskal's simplicity and cache behaviour pay off.

One practical note: the original SNAP roadNet-PA file is not strongly connected (it has a few small islands). For the whole graph, Kruskal's algorithm produces a minimum spanning forest (one tree per connected component). Prim's algorithm, starting from a fixed source, spans only the source's component, producing a single tree covering 1,087,562 vertices rather than a forest of the entire graph. This explains why the two total weights differ (1,087,561 vs 1,087,886): they refer to different subgraphs. For a connected graph, both algorithms would give the same total weight.

Both handle disconnected inputs gracefully. Prim simply finishes when its heap empties (after covering only its component), and Kruskal finishes when no more edges can be added without forming a cycle (covering every component). Thus, they do not produce the same total weight on a disconnected graph; that equality holds only when the graph is connected.

8. Reflection

Working through all four algorithms one after another helped me see how they relate to each other much better than just reading about them. DFS and BFS are built in almost the same way, using a colour array, a parent array, and a frontier. The main difference is that DFS uses the call stack as its frontier, while BFS uses a FIFO queue. Once I understood that, Prim's algorithm felt like a natural next step. It uses the same basic structure, but with a priority queue as the frontier and edge weights deciding the order.

The most important thing I learned was how tricky tie-breaking can be. When I first tested my code on wGraph1.txt, I thought Prim and Kruskal would give me the same set of 12 edges. Instead, Prim chose E–F and Kruskal chose D–E. Both results are valid minimum spanning trees with a total weight of 16. When I looked closer, I saw that Prim's choices depend on what is in the heap at each step, while Kruskal's choices depend only on the sorted order of all edges. Seeing two correct answers to the same problem reminded me that there is not always just one MST.

A year ago, I would have just used a simple boolean visited array instead of the WHITE/GREY/BLACK colouring from Cormen for DFS and BFS. Using all three states this time gave me a much clearer idea of what it means for a node to be "currently being processed," and it made it easy to track discovery and finish times. Now I understand why the textbook uses this approach: it works well for other algorithms like topological sort and SCC, where finish times are important.

Writing path compression in union-find was the most satisfying part for me. It only takes one line inside the find() function, but it makes a big difference to how fast Kruskal's algorithm runs. When you combine it with union by rank, each operation is basically constant time. Without path

compression, a long chain of unions in a worst-case graph would make every find operation walk through the whole chain.

Running the algorithms on the real-world roadNet-PA graph taught me another important lesson: theory often assumes graphs are connected, but real-world data can be disconnected. At first, I thought Prim and Kruskal would yield the same total weight, but they didn't because Prim only covered the start vertex's component, while Kruskal built a forest spanning the whole graph. This showed me that understanding what an algorithm actually does, not just what it's supposed to do in textbook examples, is crucial when working with real-world data.

If I were to do this assignment again, I would use my own indexed MinHeap class with a working decreaseKey for Prim's algorithm instead of java.util.PriorityQueue. The PriorityQueue is shorter to use, but it adds duplicate entries instead of updating keys, so the heap can grow to hold up to E entries even though only V are needed. This is not a problem for small graphs like wGraph1, but on the larger SNAP road network, it made Prim's memory use much higher. Building the proper indexed heap would also match what Cormen suggests.

Overall, this assignment helped me move from just understanding these algorithms in theory to actually implementing, debugging, and analyzing them. It also showed me why two correct implementations can give different but equally valid results.